```java
import java.util.*;
import java.util.stream.Collectors;
public class StreamAPI {
    public static void main(String[] args) {
        List<Integer> l1 = new ArrayList<>(Arrays.asList(50, 10, 30, 20, 65));
        //filter()
        List<Integer> res=l1.stream()
                            .filter(n -> n%2==0)
                            .sorted()
                            .toList();
        System.out.println(res);

        //map()
        List<Integer> res1 = l1.stream()
                           .map(n -> n*n)
                           .toList();
        System.out.println(res1);

        //square even number from list
        List<Integer> res2=l1.stream()
                          .filter(n -> n%2==0)
                          .map(n -> n*n)
                          .toList();
        System.out.println(res2);

        //count how many numbers are greater than 20
        long count = l1.stream()
                       .filter(n -> n > 20)
                       .count();
        System.out.println("Count of numbers greater than 20: " + count);

        //find the sum from a list
        int sum = l1.stream()
                    .reduce(0,Integer::sum);
        System.out.println(sum);

        //find the product from a list
        int prod = l1.stream()
                     .reduce(1, (a,b) -> a*b);
        System.out.println(prod);

        //find sum of even numbers in a list
        int sumEven = l1.stream()
                        .filter(n -> n%2==0)
                        .reduce(0,Integer::sum);
        System.out.println(sumEven);

        //sum of squares of even numbers in a list
```

```
        int sumOfSquares = l1.stream()
                             .filter(n -> n%2==0)
                             .map(n -> n*n)
                             .reduce(0,Integer::sum);
        System.out.println("Sum of squares of even numbers: " + sumOfSquares);

        //find the maximum number from a list
        Optional<Integer> max = l1.stream() //why optional because may be
stream is empty, so we need to handle that case
                                  .max(Integer::compareTo);
        System.out.println("Maximum number: " +
max.orElse(null));  //max.get() will throw exception if stream is empty, so we
use orElse() to handle that case

        //find the minimum number from a list
        Optional<Integer> min = l1.stream()
                                  .min(Integer::compareTo);
        System.out.println("Minimum number: " + min.orElse(null));

        //find first number greater than 20
        Optional<Integer> res3 = l1.stream()
                                  .filter(n -> n > 20)
                                  .findFirst();
        System.out.println(res3.orElse(-1)); //if no number greater than 20,
return -1

//**************************************************************************
**********************************************************
        //find all distinct elements from a list
        List<Integer> nums = new ArrayList<>(Arrays.asList(10, 20, 30, 10, 20,
40, 50));
        List<Integer> distinctNums = nums.stream()
                                         .distinct()
                                         .toList();
        System.out.println(distinctNums);

        //find the average of all numbers in a list
        double avg= nums.stream()
                        .mapToInt(Integer::intValue)
                        .average()
                        .orElse(0.0);
        System.out.println("Average: " + avg);

        //find the sum of all numbers in a list
        int totalSum = nums.stream()
                           .mapToInt(Integer::intValue) //intermediate
operation to convert Integer to int
                           .sum(); //terminal operation to find the sum
```

```
        System.out.println("Total sum: " + totalSum);

       //find maximum and minimum number from a list
        OptionalInt maxNum = nums.stream()
                                 .mapToInt(Integer::intValue) //intermediate
operation to convert Integer to int
                                 .max(); //terminal operation to find the
maximum
        OptionalInt minNum = nums.stream()
                                 .mapToInt(Integer::intValue)
                                 .min();
        System.out.println("Maximum number: " + maxNum.orElse(-1));
        System.out.println("Minimum number: " + minNum.orElse(-1));

        //sort asc and desc order using stream
        List<Integer> sortedAsc = nums.stream()
                                      .sorted()
                                      .toList();
        List<Integer> sortedDesc = nums.stream()
                                       .sorted(Comparator.reverseOrder())
                                       .toList();
        System.out.println("Sorted in ascending order: " + sortedAsc);
        System.out.println("Sorted in descending order: " + sortedDesc);

        //check all numbers are positive
        boolean allPositive = nums.stream()
                                  .allMatch(n -> n>0);
        System.out.println("All numbers are positive: " + allPositive);

        //check if any number is divisible by 5
        boolean anyDivisibleBy5 = nums.stream()
                                      .anyMatch(n -> n%5==0);
        System.out.println("Any number divisible by 5: " + anyDivisibleBy5);

        //count how many strings start with a specific letter 'A'
        List<String> names = new ArrayList<>(Arrays.asList("Alice", "Bob",
"Charlie", "David", "Eve", "Frank", "Grace", "Hannah", "Ivy", "Jack", "Kathy",
"Liam", "Mia", "Noah", "Olivia", "Paul", "Quinn", "Rachel", "Sam", "Tina",
"Uma", "Victor", "Wendy", "Xander", "Yara", "Zoe"));
        long countNames = names.stream()
                               .filter(name -> name.startsWith("A"))
                               .count();
        System.out.println("Number of names starting with 'A': " +
countNames);

        //join all strings in a list into a single comma-separated string
        String joinedNames = names.stream()
                                  .filter(name -> name.startsWith("A"))
```

```
                                    .collect(Collectors.joining(", "));
        System.out.println(joinedNames);

        //flatten the list of lists into a single list
        List<List<Integer>> listOfLists = new ArrayList<>(Arrays.asList(

            Arrays.asList(1, 2, 3),
            Arrays.asList(4, 5, 6),
            Arrays.asList(7, 8, 9)
        ));
        List<Integer> flattenedList = listOfLists.stream()
                                                 .flatMap(List::stream)
                                                 .toList();
        System.out.println(flattenedList);

        //find the second highest number in a list
        int secondHighest =  nums.stream()
                                 .sorted(Comparator.reverseOrder())
                                 .distinct()
                                 .skip(1)
                                 .findFirst()
                                 .orElse(-1);
        System.out.println("Second highest number: " + secondHighest);
//**************************************************************************
********************************************************** */
        //sort list of employees by salary in ascending order using stream
        List<Employee> employees = new ArrayList<>();
        employees.add(new Employee("Alice", 30, 50000));
        employees.add(new Employee("Bob", 25, 60000));
        employees.add(new Employee("Charlie", 35, 45000));

        List<Employee> sortedEmployees = employees.stream()
                                                  .sorted(Comparator.comparin
g(Employee::getSalary)) //custom sorting using comparator
                                                  .toList();
        System.out.println(sortedEmployees);

        //calculate the average age of employees using stream
        double avgAge = employees.stream()
                                 .mapToInt(Employee::getAge) //intermediate
operation to convert Employee age to get age int
                                 .average() //terminal operation to calculate
average
                                 .orElse(0.0); //terminal operation to handle
empty stream case
        System.out.println("Average age: " + avgAge);

        //find the employee with the highest salary using stream
```

```java
        Employee highestPaidEmployee = employees.stream()
                                                .max(Comparator.comparing(Empl
oyee::getSalary)) //custom logic using comparator like custom sorting ,custom
max/min
                                                .orElse(null);
        System.out.println("Employee with highest salary: " +
highestPaidEmployee);

        //group employees by age using stream => divides data into multiple
groups based on a key/category.
        Map<Integer, List<Employee>> employeesByAge = employees.stream()
                                                               .collect(Colle
ctors.groupingBy(Employee::getAge));
        System.out.println("Employees grouped by age: " + employeesByAge);

        //group employees by salary using stream
        Map<Double, List<Employee>> employeesBySalary = employees.stream()
                                                                 .collect(Col
lectors.groupingBy(Employee::getSalary));
        System.out.println("Employees grouped by salary: " +
employeesBySalary);

        //group employees by name using stream
        Map<String, List<Employee>> employeesByName = employees.stream()
                                                               .collect(Colle
ctors.groupingBy(Employee::getName));
        System.out.println("Employees grouped by name: " + employeesByName);

        //group employees by age and calculate average salary
        Map<Integer, Double> avgSalaryByAge = employees.stream()
                                                       .collect(Collectors.gro
upingBy(Employee::getAge, Collectors.averagingDouble(Employee::getSalary)));
        System.out.println("Average salary by age: " + avgSalaryByAge);

        //count occurence of each element in alist
        List<String> fruits = new ArrayList<>(Arrays.asList("apple", "banana",
"orange", "apple", "banana", "kiwi"));
        Map<String, Long> fruitCount = fruits.stream()
                                             .collect(Collectors.groupingBy(fr
uit -> fruit, Collectors.counting()));
        System.out.println("Fruit count: " + fruitCount);

        //partition employees based on salary greater than 50000 => divides
data into 2 groups: true and false.
        Map<Boolean, List<Employee>> partitionedEmployees = employees.stream()
                                                                     .collect(
Collectors.partitioningBy(emp -> emp.getSalary() > 50000));
```

```
        System.out.println("Partitioned employees: " +
partitionedEmployees.get(true)); //employees with salary greater than 50000
        System.out.println("Partitioned employees: " +
partitionedEmployees.get(false)); //employees
    }
}
class Employee{
    private String name;
    private int age;
    private double salary;

    public Employee(String name, int age, double salary) {
        this.name = name;
        this.age = age;
        this.salary = salary;
    }

    public String getName() {
        return name;
    }

    public int getAge() {
        return age;
    }

    public double getSalary() {
        return salary;
    }

    @Override
    public String toString() {
        return "Employee{" +
                "name='" + name + '\'' +
                ", age=" + age +
                ", salary=" + salary +
                '}';
    }
}
```